Seu trabalho, com um próximo passo.

Material de apoio para apresentar o projeto em uma entrevista. Guilherme Zalla - desenvolvimento full stack. Revisão: 12/11/2026.

Apresentação em 30 segundos

O TaskFlow é uma aplicação web para organizar projetos e tarefas em equipe. Ele reúne prioridades, responsáveis, prazos e progresso em um mesmo ambiente. Desenvolvi uma interface em React e TypeScript, integrada a uma API em C# com ASP.NET Core e MySQL. O foco é transformar o planejamento em um fluxo de trabalho claro, do cadastro da tarefa até a conclusão.

Que problema ele resolve?

Quando tarefas, datas e decisões ficam espalhadas, a equipe perde contexto e não sabe o que fazer primeiro. O TaskFlow organiza o trabalho em espaços, projetos e tarefas, com filtros e visões complementares para acompanhar as entregas.

O que mostrar primeiro

Comece pela visão geral: explique que as contagens e o progresso são calculados a partir dos registros. Abra Projetos e depois Tarefas. Mostre lista e quadro e use uma tarefa para explicar prioridade, status e prazo.

Público e formato

O projeto atende à organização de trabalho colaborativo. É uma aplicação web responsiva com interface em português e inglês. Ainda não possui aplicativo nativo. A demonstração usa dados fictícios armazenados no navegador e é apresentada como um modo separado da API real.

Como o TaskFlow funciona

Qual é a estrutura dos dados?

Um espaço de trabalho reúne projetos. Cada projeto reúne tarefas. As tarefas registram título, descrição, prioridade, status, responsável e prazo quando definidos. Participantes e permissões pertencem ao contexto do espaço.

Como alguém usa no dia a dia?

A pessoa entra na conta, cria ou acessa um espaço, organiza seus projetos e cadastra tarefas. A lista facilita filtrar e editar. O quadro agrupa as tarefas em A fazer, Em andamento, Em revisão e Concluída. Calendário e relatórios oferecem outras leituras das mesmas informações.

Quais recursos já existem?

Cadastro e login; espaços, projetos e tarefas; equipes e atividade; comentários e menções; notificações internas e lembretes; relatórios com exportação CSV; etiquetas, dependências e modelos de projeto. O escopo pode ser demonstrado sem inventar horas trabalhadas ou índices de produtividade.

O que diferencia o modo de demonstração?

Ele utiliza um adaptador local, com registros fictícios e persistência no navegador. Permite conhecer o produto sem criar conta ou configurar o banco. Ele não comprova autenticação, permissões, envio de e-mail nem integração MySQL.

Como os idiomas funcionam?

Um catálogo de tradução cobre a interface em português e inglês. A preferência é salva no navegador e enviada como Accept-Language. Conteúdo criado pelo usuário não é traduzido automaticamente; as respostas próprias da API ainda exigem validação de idioma.

Arquitetura e decisões

CAMADA	RESPONSABILIDADE
React + TypeScript	Interface, rotas, formulários e estados visuais.
TanStack Query + Axios	Consultas, cache, invalidação e comunicação HTTP.
API / Application	Endpoints, comandos, validação e execução de casos de uso.
Domain / Infrastructure	Entidades; EF Core, Identity e persistência no MySQL.

Por que separar frontend e backend?

A API concentra persistência e regras de acesso. O frontend organiza a experiência de uso. A separação permite evoluir as duas partes e explicitar o contrato HTTP, mas exige configurar CORS e tratar falhas de comunicação.

Como funciona a autenticação?

A API usa ASP.NET Core Identity. O frontend envia as credenciais a /login, armazena o accessToken retornado e o envia no cabeçalho Bearer. O cadastro com nome usa /api/auth/register. A proteção das rotas no navegador orienta a navegação; a autorização efetiva precisa ocorrer na API.

Como os dados são atualizados na tela?

TanStack Query controla consultas e cache. Depois de uma alteração, a interface invalida as consultas relacionadas para atualizar listas, dashboard e relatórios. O Axios centraliza o endereço da API, os cabeçalhos e o tratamento de erros.

Respostas diretas, com limites claros

Como você controla o acesso aos dados?

Os endpoints protegidos exigem autenticação. As consultas e operações verificam o usuário e seu vínculo com o espaço, conforme a operação. É preciso demonstrar com testes que outro usuário não acessa ou modifica recursos sem permissão; a sessão visual do frontend não é uma barreira de segurança.

Como funcionam dependências de tarefas?

O backend verifica pré-requisitos e bloqueia relações cíclicas. Isso permite representar uma ordem de execução sem permitir que uma tarefa dependa direta ou indiretamente dela mesma. A validação desses fluxos integrados permanece parte do fechamento da versão 1.0.

O que os testes comprovam hoje?

Há testes de regras do frontend e testes .NET de validação e integração HTTP. A fábrica dos testes de integração substitui o banco pelo provedor EF InMemory: ela não comprova comportamento transacional ou concorrência do MySQL. Não apresento os testes como cobertura total do produto.

O projeto está pronto para produção?

O núcleo do produto está implementado, mas o lançamento 1.0 ainda tem pendências. Faltam a rodada final com API/MySQL, regressão visual após o refinamento, configuração de produção, HTTPS, backups e procedimento de recuperação. E-mail e notificações externas precisam ter o escopo definido.

O que você melhoraria em seguida?

Primeiro validaria os fluxos reais de colaboração, autenticação e planejamento. Depois fecharia os testes de autorização e falhas relevantes, recuperação de senha por e-mail e a operação de produção. Priorizaria estabilidade antes de acrescentar novas telas.

Uma conversa de cinco minutos

1. Contexto e visão geral - 1 minuto

Apresente o problema, o público e o formato web. Abra a visão geral e mostre que os números vêm das tarefas e projetos. Se estiver usando a demonstração, avise desde o início que os dados são fictícios e locais.

2. Projeto e tarefa - 2 minutos

Abra um projeto, filtre suas tarefas e alterne entre lista e quadro. Crie uma tarefa, defina prioridade e projeto, salve e mude seu status. Explique como prazos e conclusão alimentam o restante da interface.

3. Colaboração e acompanhamento - 1 minuto

Mostre equipes e atividade. Explique comentários, menções e notificações internas a partir dos recursos implementados. Diferencie uma ação demonstrada de um fluxo ainda pendente de validação real.

4. Arquitetura e próximos passos - 1 minuto

Explique a divisão React, API em C# e MySQL. Cite uma decisão técnica e sua consequência. Termine com as pendências reais para 1.0, sem prometer recursos de produção que ainda não foram entregues.

Se você não souber responder

Diga o que sabe, indique o limite e explique como verificaria. Exemplo: "A regra está implementada, mas eu ainda não validei essa condição concorrente no MySQL. Eu montaria um cenário isolado com duas requisições simultâneas antes de afirmar que está coberto."

Como explicar o uso de IA

As ferramentas foram utilizadas como apoio à documentação, referências visuais, refinamento de interface e elaboração e revisão de testes. A autoria do produto, as regras de negócio, as decisões de arquitetura e a responsabilidade pelo resultado são do desenvolvedor.